

Reviewer Pack — Minimal Symmetry of Quiver Representations in Resolution Low...

Entry 06856744ef95 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 18:42:17 UTC

Minimal Symmetry of Quiver Representations in Resolution Lower Bounds

Entry ID: 06856744ef95 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 18:42:17 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Resolution Proof Complexity

Statement:

[‘For any quiver representation Q with symmetry group G , the Tseitin formula constructed from Q has a resolution refutation of length $\geq 2^{\Omega(|G|)}$, where $|G|$ is the number of conjugacy classes in G .’, ‘Equivalently, for a given quiver Q , there exists an upper bound on the resolution proof length as a function of the size of its symmetry group.’, ‘Moreover, this bound holds true for all non-linearly equivalent quivers.’]

Rationale (proposer’s reasoning):

[‘Algebraic combinatorics provides a rich framework to study symmetries in mathematical structures. Exploring these symmetries may uncover inherent complexity properties that are not easily observed through other means.’, ‘The connection between the symmetry of quiver representations and resolution proof length could provide new insights into proving lower bounds for Tseitin formulas, potentially breaking current barriers in resolution complexity.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1d1502ef88425fdd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all quivers Q , the mean resolution refutation length of their Tseitin formulas is at least $2^{\Omega(|G|)}$ where $|G|$ is the number of conjugacy classes in G ’s symmetry group, and falsified if any seed produces a mean resolution refutation length below this threshold.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	UNCERTAIN	1.00	SAFE	HITS
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quiver representation" AND "resolution proof complexity" AND Tseitin formula" - "symmetry group G" AND resolution refutation" AND length $\geq 2^{\Omega(|G|)}$ " - "upper bound on resolution proof length" AND function of symmetry group size"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_edge(v, w):
        return (v, w) in quiver or (w, v) in quiver

    def find_symmetry_group():
        n = len(quiver)
        G = []
        for perm in itertools.permutations(range(n)):
            if all(is_edge(v, w) == is_edge(perm[v], perm[w]) for u in range(n) for v, w in quiver):
                G.append(perm)
        return G

    def tseitin_formula(quiver):
        n = len(quiver)
        literals = {v: f'x{v}' for v in range(n)}
        clauses = []
        for u in range(n):
            for v, w in quiver[u]:
                clauses.append([literals[v], literals[w]])
                clauses.append([-literals[v], -literals[w]])
                clauses.append([-literals[v], literals[w]])
                clauses.append([literals[v], -literals[w]])
        return clauses
```

```

def resolution_refutation_length(clauses):
    n = len(quiver)
    unit_clauses = {l: [] for l in range(-n, n+1)}
    for clause in clauses:
        for literal in clause:
            unit_clauses[literal].append(clause)

    resolvents = []
    while True:
        new_resolvents = set()
        for clause in clauses:
            if len(clause) == 1:
                resolvents.append(clause[0])
                continue
            for literal in clause:
                if -literal in resolvents:
                    new_resolvent = [l for l in clause if l != literal and -l != literal]
                    if new_resolvent not in new_resolvents:
                        new_resolvents.add(tuple(sorted(new_resolvent)))
        if not new_resolvents:
            break
        clauses.extend(new_resolvents)

    return len(resolvents)

n = random.randint(5, 40)
quiver = {v: [] for v in range(n)}
for u in range(n):
    for v in range(u+1, n):
        if random.choice([True, False]):
            quiver[u].append((u, v))
            quiver[v].append((v, u))

symmetry_group = find_symmetry_group()
resolution_length = resolution_refutation_length(tseitin_formula(quiver))

return {
    "metric_name": "resolution_refutation_length",
    "metric_value": resolution_length,
    "instances_tested": 1,
    "conjecture_holds": resolution_length >= 2**(math.log(len(symmetry_group), 2)),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_length} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_length} std={math.sqrt(sum((r['metric_value'] - mean_length)**2 for r in results))}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='resolution_length < 2^(log(|G|, 2))' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm or refute the conjecture based on the results obtained. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15732
2	preregistration	ollama_remote	glm4:latest	0	0	9218
3	novelty	ollama_remote	glm4:latest	0	0	9932
4	novelty	ollama_remote	glm4:latest	0	0	8835
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16003
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9466
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17455
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10639
9	judge	ollama_remote	glm4:latest	0	0	11162

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 108442 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/06856744ef95.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/06856744ef95.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/06856744ef95.tar.gz` (if generated)